

MEDGUARD

AI Powered Medication Safety Assistant

A Multi-Task Clinical NLP System with Knowledge-Graph Enrichment
and Conversational AI

Course: SE 4003 — Multidisciplinary Engineering Projects

Instructor: Doç.Dr. Selim Yılmaz

Student: Alaa Hosny Saber Hassouba

Student ID: 220717702

Semester: 2025–2026 Spring

Date: May 2026

GitHub: <https://github.com/Eng-AlaaHosny/MedGuard>

Muğla Sıtkı Koçman University — Faculty of Engineering, Software
Engineering

Abstract

MedGuard is an academic drug–drug interaction (DDI) detection system built on Bio_ClinicalBERT, trained with a three-stage curriculum:

Stage 1 recognises drug mentions (NER), Stage 2 classifies interaction type, and Stage 3 estimates clinical severity.

The model fuses knowledge-graph embeddings from DrugBank via node2vec (128-d per drug) and Lipinski physicochemical features (5-d per drug) alongside the text encoder, producing a 901-dimensional per-drug representation.

The system is served via FastAPI with a browser demo and an optional Claude-powered conversational assistant.

Metric	Value	Notes
NER macro-F1 (B+I only)	0.9517	Validation set; O class excluded
Interaction-type macro-F1	0.4190	Validation set; 5-class; macro-F1 only; per-class F1 not reported
Severity macro-F1	0.3602	Validation set; 4-class; macro-F1 only; per-class F1 not reported
DDInter coverage	795 / 3,411 (23.31%)	Pairs with resolved severity label

1. Introduction

1.1 The Problem

Every year, more than 1.3 million people in the United States alone are rushed to emergency rooms because of a preventable medication error, at an economic cost exceeding \$3.5 billion annually.

Of all medication errors, drug–drug interactions (DDIs) are among the most insidious — they are invisible until they strike.

A DDI occurs when two or more drugs interact in ways that neither would alone:

the combination can amplify a drug’s effect to toxic levels, cancel it entirely, or produce an unexpected adverse reaction. Warfarin combined with aspirin dramatically increases bleeding risk; amiodarone with digoxin can cause cardiac toxicity; ciprofloxacin with warfarin raises anticoagulant levels through metabolic pathway interference.

The problem is sharpest for the most vulnerable. Elderly patients often manage five to ten medications prescribed by multiple specialists — a cardiologist, endocrinologist, neurologist, and general practitioner — each seeing only their own slice of the patient’s medication profile.

No single person holds the full picture. Consumer-facing apps generate so many low-priority alerts that clinicians begin to ignore them (alert fatigue).

Reference databases like DrugBank are comprehensive but designed for pharmacologists, not patients.

Statistic	Value
Annual US emergency room visits from adverse drug events	1.3 million
Annual economic cost of preventable medication errors	\$3.5 billion+
Typical medication load for elderly patients with chronic conditions	5–10 drugs
Hospital readmissions linked to medication-related problems	~30%

1.2 What MedGuard Solves

MedGuard targets three groups currently underserved by existing tools:

- Patients managing multiple prescriptions — particularly elderly individuals or those with chronic conditions — who need to know whether a new prescription is safe alongside what they already take.
- Caregivers (family members or home-care workers) managing medications on behalf of someone else, who need clear, actionable safety information without requiring pharmacology training.
- Pharmacy students and junior clinicians who benefit from an educational tool that explains not just the interaction result but the reasoning behind it: mechanism, severity rationale, and relevant clinical literature.

MedGuard addresses this by combining four capabilities that no existing consumer tool currently offers together:

- Natural Language Understanding — Users submit two drug names (with optional clinical context text); NER highlights drug mentions in the displayed sentence.
- Risk Stratification — assigns a severity level (safe, caution, warning, danger) trained on DDInter’s curated risk database and expressed in plain colour-coded language.
- Explainability — explains the result through pharmacokinetic or pharmacodynamic mechanism descriptions pulled from DrugBank’s knowledge graph.
- Multi-Modal Intelligence — fuses clinical language (Bio_ClinicalBERT), relational drug knowledge (DrugBank knowledge graph with node2vec embeddings), and molecular chemistry (Lipinski physicochemical features) to combine complementary signals from text, graph structure, and physicochemical descriptors (ablation across single modalities was not performed in this project).

1.3 Scope and Limitations

MedGuard is an academic capstone project developed for SE 4003 at Muğla Sıtkı Koçman University (2025–2026 Spring). It delivers an implemented and evaluated AI core — a trained model, a REST API, and a browser demo — alongside an honest account of its limitations. A full patient-facing React application (with profile management and daily medication scheduler) remains future work; deploying a production patient application on top of an unvalidated model would be irresponsible. The current phase establishes the foundation on which a patient-safe product can be built.

2. Related Work

DDI detection from text is a well-established NLP research problem, anchored by the SemEval-2013 Task 9 shared task and the DDI Corpus it produced. Most prior academic work stops at the NER and classification level. Few systems integrate structural pharmacological knowledge (knowledge graphs) and molecular chemistry (physicochemical features) in a single model.

Fewer still expose their results through an interactive API and a conversational AI interface accessible to non-researchers. MedGuard is designed to bridge that gap.

Reference	Contribution to MedGuard
Bio_ClinicalBERT (Alsentzer et al., 2019)	BERT pre-trained on MIMIC-III clinical notes; used as shared encoder
DDI Corpus / SemEval-2013 Task 9 (Herrero-Zazo et al., 2013)	Sentence-level annotated drug pairs; Stages 1–2 training data
node2vec (Grover & Leskovec, 2016)	Graph embeddings via biased random walks; KG feature extraction
DDInter (Xiong et al., 2022)	Curated drug-pair severity risk levels; Stage 3 training labels
Lipinski Rule of Five (Lipinski et al., 1997)	Five physicochemical drug descriptors; side-feature enrichment
DrugBank 5.0 (Wishart et al., 2018)	Drug names, synonyms, interactions, mechanism text; seeds knowledge graph
Balanced Softmax / Logit Adjustment (Menon et al., 2021)	Long-tail loss function for sparse severity classes

3. System Architecture

MedGuard has two phases: an offline build-and-train phase and an online inference phase. Understanding this split is the key to the whole system — the offline phase produces all the artefacts (knowledge graph, embeddings, checkpoints) that the online phase consumes at request time.

3.1 Offline Phase

The offline phase preprocesses raw data and trains the model. Each script has a single responsibility:

- `preprocessor.py` — parses DDI Corpus XML into `DDISentence` objects.
- `drugbank_processor.py` — builds `drugbank.db` (drugs, interactions, synonyms).
- `kg_builder_full.py` — constructs the NetworkX knowledge graph and computes node2vec embeddings → `knowledge_graph.pkl`.
- `ddinter_vocabulary.py` — loads DDInter CSVs into a normalised name table.
- `trainer.py` — runs the three-stage curriculum pipeline, producing `.pt` checkpoints.

3.2 Online Phase

At startup, main.py loads the tokeniser, two model checkpoints, the knowledge graph, and Lipinski data. FastAPI serves two route files:

- routes.py — core DDI inference (POST /api/analyze).
- assistant_routes.py — Claude-powered conversational assistant (POST /api/assistant/chat).

The browser demo at demo.html calls POST /api/analyze and renders colour-coded severity badges.

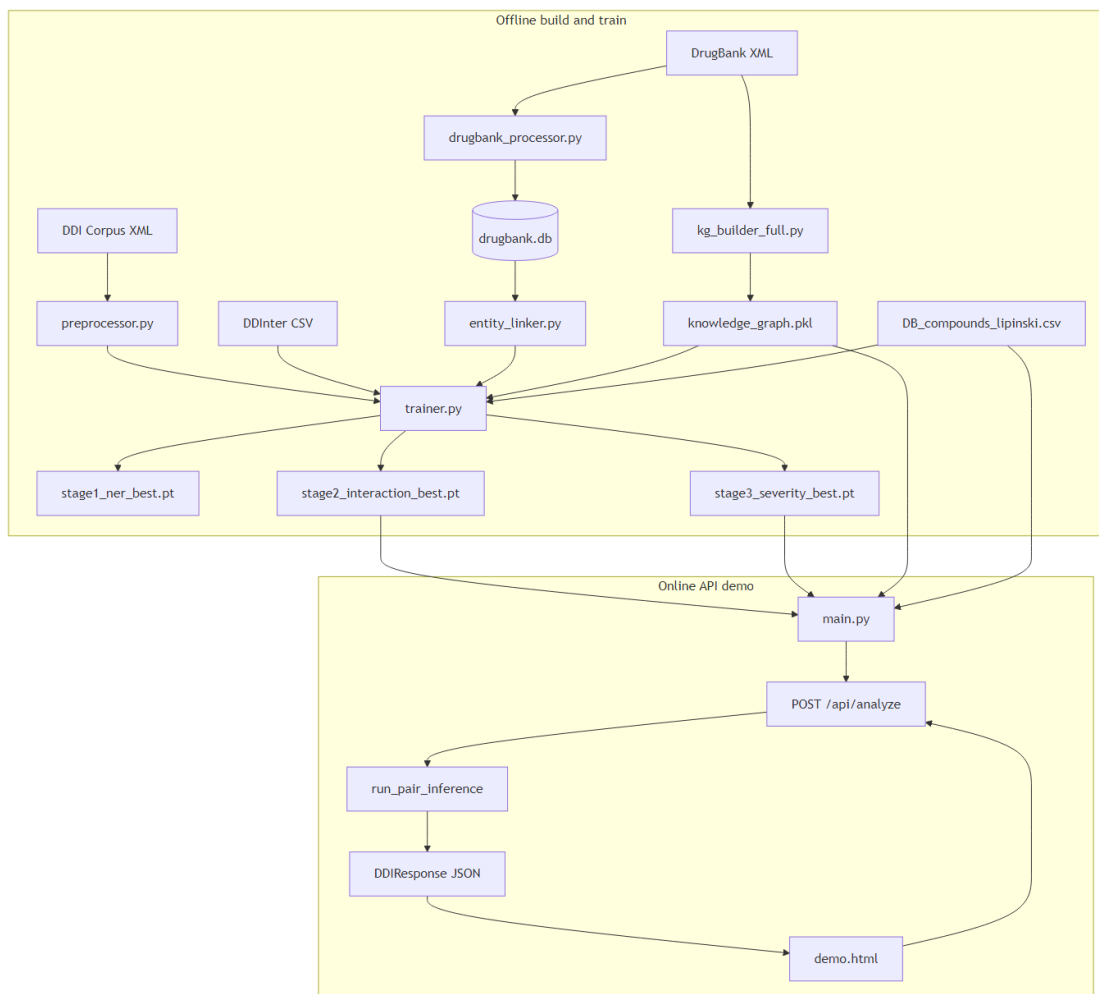


Figure 1. MedGuard system overview: offline data builders and three-stage training produce checkpoints and knowledge_graph.pkl; the online phase loads these artefacts in main.py and serves POST /api/analyze to demo.html.

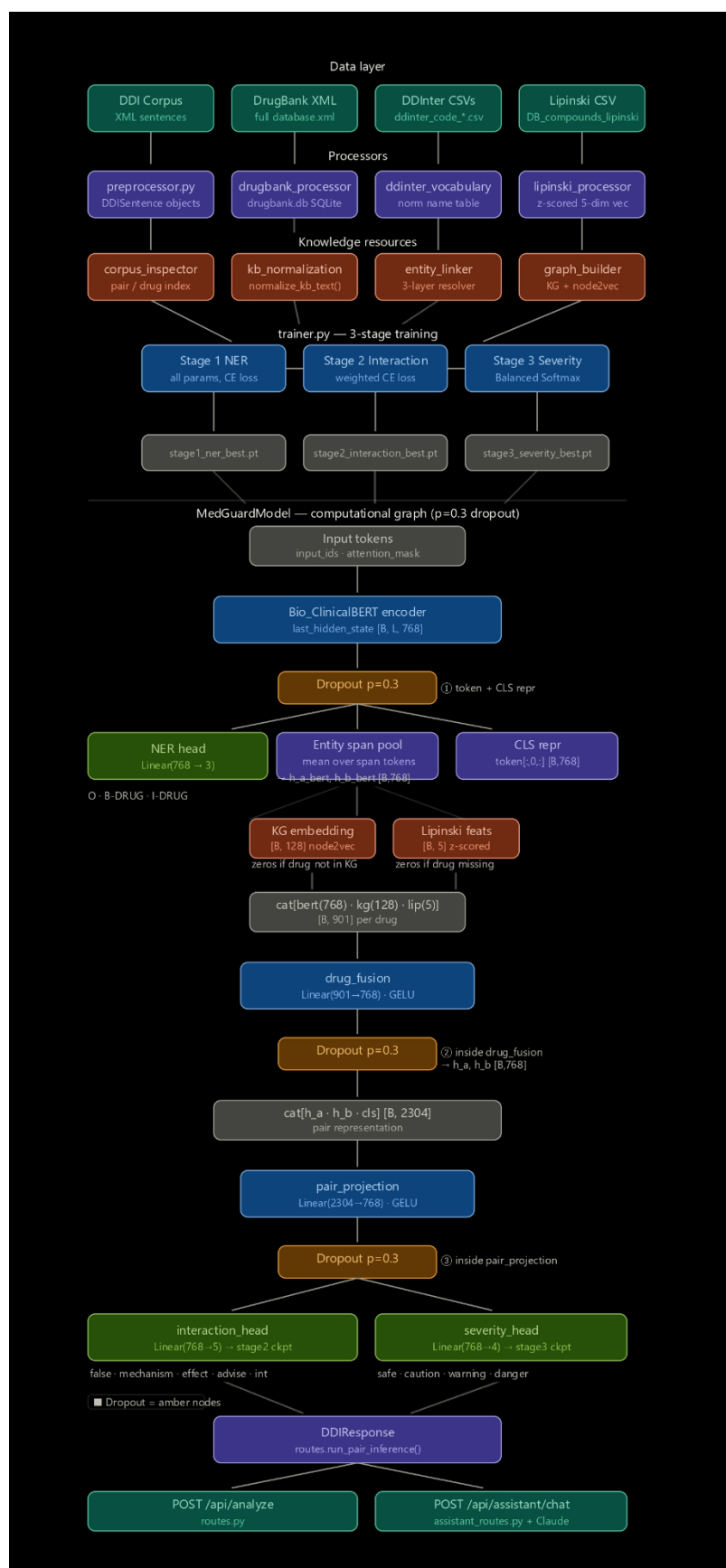


Figure 2. Data layer, processors, and training pipeline: DDI Corpus, DrugBank, DDinter, and Lipinski feed preprocessors and builders that produce drugbank.db, knowledge_graph.pkl, and stage checkpoints consumed at inference.

4. Data Sources & Preprocessing

Before the model can be understood, the data that feeds it must be understood. MedGuard draws on six primary data sources (plus derived artefacts such as `drugbank.db`, `knowledge_graph.pkl`, and training checkpoints), each serving a different role in the pipeline. Text normalisation for all knowledge-base lookups is handled by `normalize_kb_text()` in `kb_normalization.py`: lower-casing, removing parenthetical content, stripping punctuation, and removing salt-form words (hydrochloride, sodium, etc.) so that “Warfarin sodium” and “Warfarin” resolve to the same key.

Dataset / File	Contents	Used For
DDI Corpus (XML)	Clinical sentences, drug entity spans, interaction type labels	Stages 1–2 training; Stage 3 sentences
DrugBank XML	Drug names, synonyms, interactions, mechanism text	Build <code>drugbank.db</code> ; seed knowledge graph
<code>drugbank.db</code> (SQLite)	Drugs, synonyms, interactions, DDInter vocab	Linking, Lipinski ID lookup, API KB context
<code>ddinter_code_*.csv</code>	Curated pairs with risk level (minor / moderate / major)	Stage 3 severity labels
<code>DB_compounds_lipinski.csv</code>	5 physicochemical numbers per compound	Stages 2–3 training; inference side features
<code>knowledge_graph.pkl</code>	NetworkX graph + node2vec embeddings + name maps	128-d drug features; KG context in API
Checkpoints (.pt)	Saved neural network weights per stage	Inference

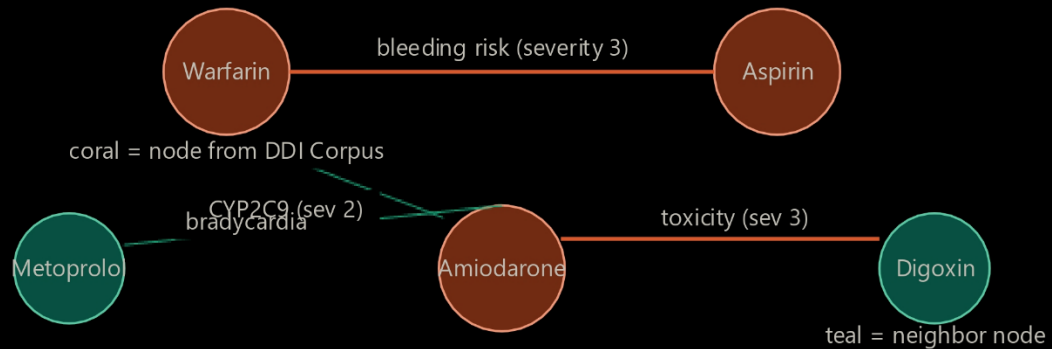
5. Knowledge Graph Construction

`DrugKnowledgeGraph` (`graph_builder.py`) wraps a NetworkX directed graph where each node is a drug (DrugBank ID) and each edge is a known interaction carrying a severity score (0–3) and description text. The graph is seeded with DDI Corpus drugs; their DrugBank interaction neighbours are added automatically.

`node2vec` is configured with 200 random walks per node of length 30, window 10, and 128-dimensional output (KG_DIM). Drugs with similar interaction profiles are geometrically close in embedding space. At inference, embeddings are retrieved by normalised name; drugs absent from the graph receive a zero vector and `modality_coverage.kg_embedding = false`. When both drugs have a direct edge in the graph, the edge description is returned as `kg_context` in `DDIResponse`, providing human-readable provenance alongside the neural prediction.

Knowledge graph — structure

NetworkX = a Python library for creating and querying graphs. Not ML — just a data structure.



Node = a drug (DrugBank ID + name). Edge = known interaction (from DrugBank XML).

Edge attributes: interaction_type, severity score, description text

Only DDI Corpus drugs are seeded. Their DrugBank neighbors are added automatically.

node2vec — how KG embeddings are produced

node2vec performs random walks starting from every node (200 walks, length 30).

Drugs that interact with the same neighbors end up near each other in the walk sequences.

A word2vec model is then trained on those sequences — treating drugs like words.

Result: each drug gets a 128-dim vector. Drugs with similar interaction profiles are nearby

This vector is loaded from knowledge_graph.pkl at startup and looked up by drug name at inference.

How does the KG "inform" the backbone?

It does NOT plug into BERT directly. It is concatenated AFTER BERT, inside drug_fusion.

Think of it as extra side-information: BERT reads the text, KG says "these two drugs are structurally neighbours in the interaction graph". Both signals get compressed together.

If drug not found in KG → kg_embedding = zeros([B,128]). Model still runs, just without KG signal.

Is the DDI Corpus the heart of everything?

Yes. It provides: training sentences, drug entity spans, interaction type labels (stages 1+2), and the drug names that seed the KG. DDInter provides severity labels(stage 3) via linking.

DrugBank provides the graph edges and the synonym table used by the entity linker.

Figure 3. Knowledge graph structure and node2vec embedding pipeline. Nodes are DrugBank drug entities; edges carry interaction severity and description text. node2vec walks produce 128-dimensional embeddings used as side features in the model.

6. Entity Linking

Entity linking is a prerequisite to understanding both the training data and the severity results. DictionaryFirstLinker (entity_linker.py) bridges DDI Corpus drug names to DDInter through a six-step resolver chain. The first matching step wins, and the resolution strategy at each step directly determines which training pairs receive severity labels.

Step	Strategy	Description
1	class_routing	Detects class phrases (e.g. “ACE inhibitor”). Excluded from severity training.
2	ddinter_surface	Normalised name already in ddinter_drug_names table. Direct hit.
3	DrugBank synonym	Synonym table → DrugBank ID → best matching DDInter name.
4	KG maps	Resolved via knowledge_graph.pkl, then cross-referenced to DDInter.
5	drugbank_only	Known in DrugBank but absent from DDInter. Excluded from Stage 3.
6	passthrough	Normalised raw string returned; severity lookup will likely miss.

All resolutions are logged to linking_snapshots.sqlite with a unique run_id (reference run: 854063fe93f2b781). In the reference run: 795 / 3,411 pairs (23.31%) resolved to a DDInter severity label; 539 mentions were skipped via class_routing.

This 23.31% coverage rate is the primary constraint on Stage 3 training diversity and directly explains the lower severity F1 of 0.3602. It is not a modelling failure — it is a data pipeline constraint, rooted in naming mismatches across three incompatible drug vocabularies (DDI Corpus, DrugBank, DDInter). Understanding this is essential context for interpreting the experimental results.

7. Model Architecture

With the data sources, knowledge graph, and entity linking in hand, the model architecture can be understood in full. MedGuardModel (medguard_model.py) uses Bio_ClinicalBERT (768-d hidden) as a shared encoder. The design fuses three distinct signal types per drug into a single learned representation.

7.1 Feature Fusion per Drug

For each drug, three signals are concatenated:

- BERT span mean-pool — 768-d, computed by averaging over the token positions belonging to the drug entity.
- KG node2vec embedding — 128-d per drug, looked up from knowledge_graph.pkl by normalised name.
- Lipinski physicochemical features — 5-d per drug (MW, HBA, HBD, logP, Ro5), z-scored at load time; zeros if drug not found.

Training vs inference (Stages 2–3) — important. During Stages 2 and 3, forward() does not receive drug token spans; drug-side BERT features fed into drug_fusion are taken from the sentence [CLS] embedding, while KG (128-d) and Lipinski (5-d) side features are still looked up by drug name. At inference, run_pair_inference() locates each drug's token span and mean-pools those positions to produce the 768-d BERT component passed to the same drug_fusion layers.

Because [CLS] and span-pooled vectors occupy different regions of representation space, the drug_fusion and downstream pair_projection weights were optimised under a train/serve distribution shift, not merely a minor efficiency choice. Interaction-type and severity predictions may therefore be unreliable or misleading even when NER and API behaviour appear correct. This is a first-order architectural limitation of the current system; span-consistent training (Section 12.2) is the primary corrective action.

The 901-d concatenation passes through drug_fusion: $\text{Linear}(901 \rightarrow 768) \rightarrow \text{GELU} \rightarrow \text{Dropout}(0.3)$, yielding a 768-d fused drug vector h_a or h_b .

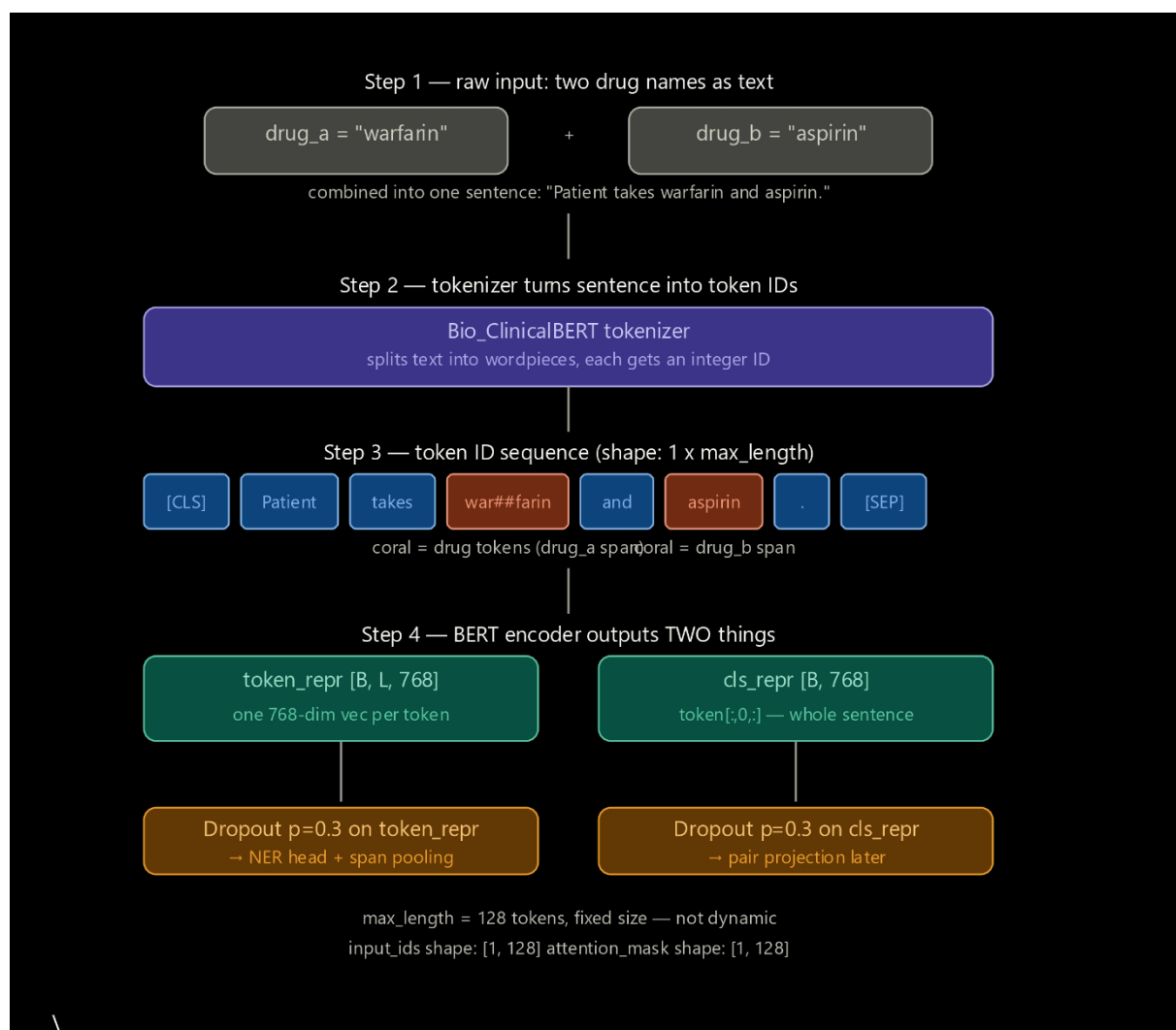


Figure 4. Input representation and backbone flow. Drug names are tokenised by Bio_ClinicalBERT (max 128 tokens); the encoder produces a token_repr tensor [B, 128, 768] and a CLS representation [B, 768].

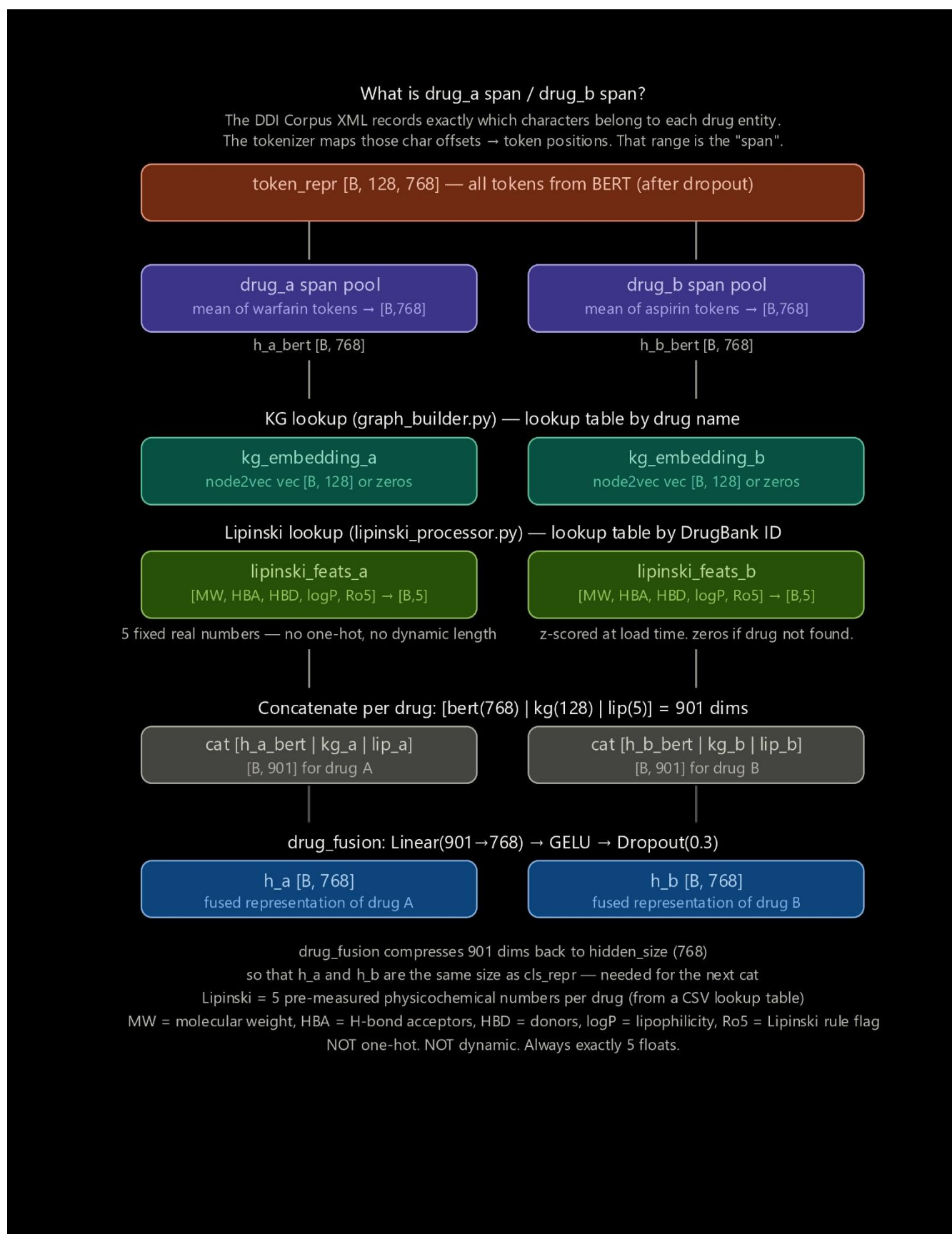


Figure 5. Drug fusion detail. Span pooling extracts h_a_bert and h_b_bert from $token_repr$; knowledge-graph embeddings (128-d) and Lipinski features (5-d) are concatenated to form the 901-d per-drug vector, then projected to 768-d via the $drug_fusion$ MLP.

7.2 Pair Representation and Classification Heads

The pair vector is $\text{cat}[h_a \mid h_b \mid \text{cls_repr}] = 2304\text{-d}$, projected through pair_projection: $\text{Linear}(2304 \rightarrow 768) \rightarrow \text{GELU} \rightarrow \text{Dropout}(0.3)$. Two classification heads read this vector:

- interaction_head — $\text{Linear}(768 \rightarrow 5)$, predicting: false / mechanism / effect / advise / int.
- severity_head — $\text{Linear}(768 \rightarrow 4)$, predicting: safe / caution / warning / danger.

A separate NER head reads token_repr directly: $\text{Linear}(768 \rightarrow 3)$ for O / B-DRUG / I-DRUG labels.

Component	Input Dim	Output Dim	Checkpoint
Bio_ClinicalBERT encoder	Token IDs (max 128)	768 per token, 768 CLS	shared
NER head	768 (per token)	3 (O/B/I)	stage3
drug_fusion MLP	901 per drug	768 per drug	stage2/3
pair_projection MLP	2304 (768×3)	768	stage2/3
interaction_head	768	5 classes	stage2
severity_head	768	4 classes	stage3

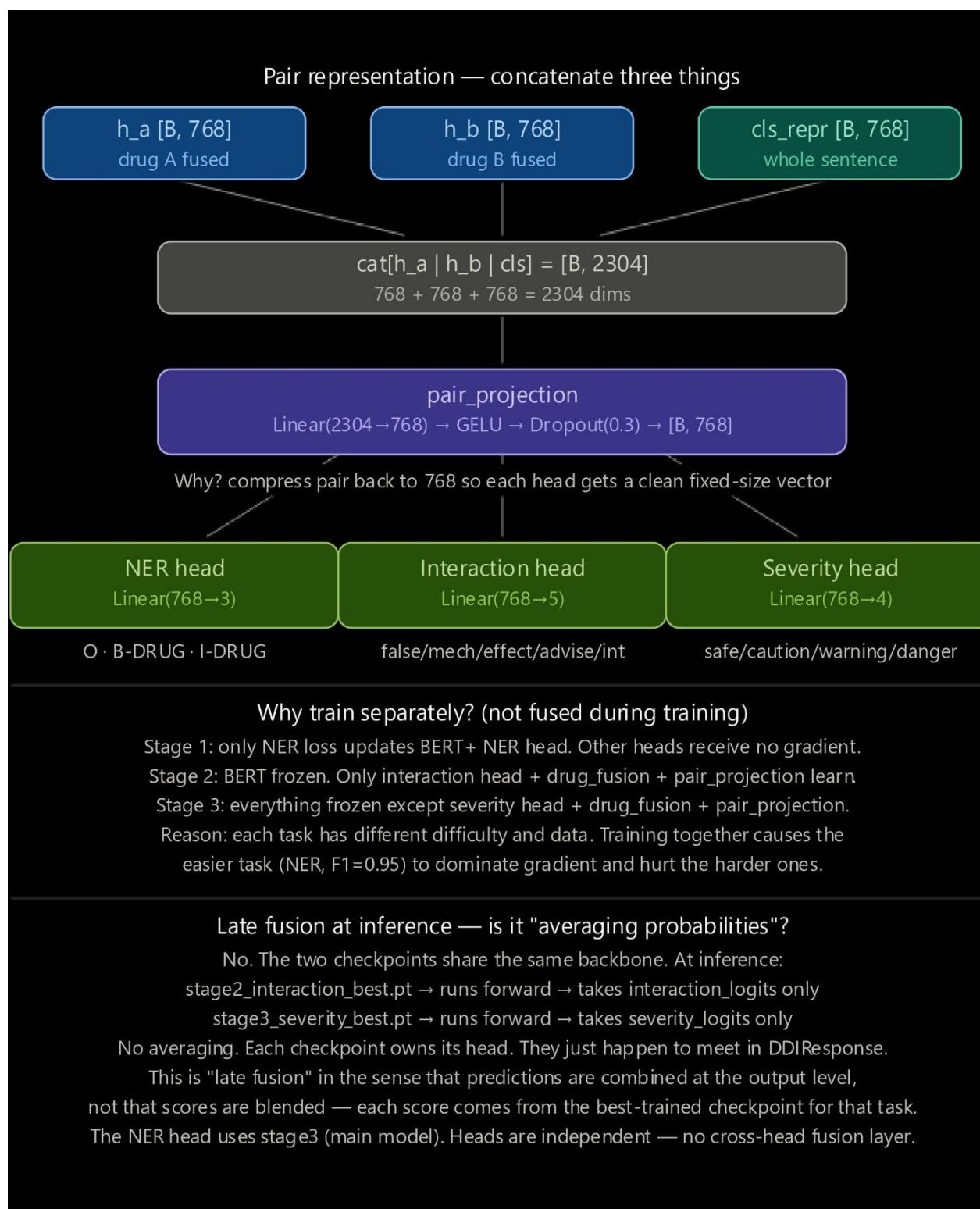


Figure 6. Pair representation, projection, and classification heads. h_a , h_b , and cls_repr are concatenated (2304-d), projected to 768-d, then fed to three independent heads: NER (token-level), interaction_head (5-class), and severity_head (4-class).

7.3 Why Separate Checkpoints?

Each task has different difficulty and data. Training all heads simultaneously lets the easiest task (NER, F1 = 0.95) dominate the gradient and damage the harder ones. The curriculum design ensures each head is optimised independently: Stage 1 trains only BERT and the NER head; Stage 2 freezes BERT and the NER head to train the interaction head; Stage 3 additionally freezes the interaction head to train the severity head.

At inference, two forward passes are run from their respective best checkpoints — there is no score blending. The stage2 checkpoint provides interaction logits; the stage3 checkpoint provides NER and severity logits.

8. Training Procedure

Reproducibility. Trained weights (stage1_ner_best.pt, stage2_interaction_best.pt, stage3_severity_best.pt) are distributed via the project Google Drive folder linked in CHECKPOINTS.md; place them under backend/app/models/checkpoints/ before running main.py.

Run with: `python -m app.models.trainer --stage all`. The three stages execute sequentially, each loading the best checkpoint from the previous stage.

- Stage 1 (NER) — All parameters trainable. Weighted CrossEntropyLoss downweights the O class. Validated on B-DRUG + I-DRUG macro-F1. Saved → stage1_ner_best.pt.
- Stage 2 (Interaction) — Encoder and NER head frozen. Trains drug_fusion, pair_projection, and interaction_head. Balanced class weights address the majority false class. Saved → stage2_interaction_best.pt.
- Stage 3 (Severity) — Interaction head additionally frozen. Trains on DDI Corpus pairs linked to DDInter severity labels via DictionaryFirstLinker. Uses BalancedSoftmaxLoss (logit adjustment) to handle the sparse caution class. Saved → stage3_severity_best.pt.

Hyperparameter	Stage 1 (NER)	Stage 2 (Interaction)	Stage 3 (Severity)
Epochs	5	10	10
Batch size	16	4	4
Learning rate	2e-5	1e-4	1e-4
Gradient accumulation	—	4	4
Loss function	Weighted CE	Weighted CE	Balanced Softmax
Validation metric	Macro-F1 (B+I)	Macro-F1	Macro-F1

9. Inference Pipeline

POST /api/analyze accepts drug_a, drug_b, and optional text. If text is empty, the backend constructs a synthetic sentence: "{drug_a} and {drug_b} may interact. Concurrent use of {drug_a} with {drug_b} should be monitored."

The text is tokenised by Bio_ClinicalBERT (max_length = 128). find_token_span() locates each drug name's token indices via offset mapping. KG embeddings and Lipinski features are looked up by drug name or DrugBank ID; missing data yields zero tensors and sets the corresponding modality_coverage flags to false.

Two forward passes are then run — one per checkpoint — and softmax produces probability distributions over class labels. KG context, Lipinski values, plain_guidance, and interaction_reason are assembled into a DDIResponse JSON object.

10. API & Conversational Assistant

10.1 FastAPI Endpoints

Method	Path	Description
POST	/api/analyze	Core DDI inference. Returns DDIResponse JSON.
GET	/api/health	Model, KG, Lipinski, and assistant readiness flags.
GET	/api/drugs	Sample drug names from the loaded knowledge graph.
POST	/api/assistant/chat	Conversational assistant (Claude + tool). Returns reply + tool_trace.

10.2 Conversational Assistant

assistant_routes.py implements an agentic loop using the Anthropic Claude API (default: claude-3-5-haiku-20241022). Claude is given a tool definition medguard_analyze_pair that calls run_pair_inference() — the same function as POST /api/analyze. When Claude emits tool_use blocks, _execute_tool() runs the inference and returns JSON as a tool_result. The loop runs up to MAX_TOOL_ROUNDS = 6.

A system prompt instructs Claude to call the tool for any two-drug query, report severity_label and interaction_type faithfully, and always include an educational-demo disclaimer. Each AssistantChatResponse includes a tool_trace for full transparency.

11. Experimental Results

Evaluation protocol. All reported metrics are macro-F1 on a held-out validation split (15% of sentences). Stage 1 reports macro-F1 on B-DRUG and I-DRUG tokens only (class O excluded). Stages 2 and 3 report macro-F1 over their respective label sets. Results are from a single training run with the hyperparameters listed below; no test-set tuning was performed.

Single-run reporting. All metrics reported here come from one full training run with fixed hyperparameters (Section 8). No repeated runs with different random seeds, cross-validation folds, or confidence intervals were computed. Given sparse Stage 3 labels (795 resolved pairs) and class imbalance, macro-F1 on Stages 2 and 3 could plausibly vary by ± 0.05 or more across seeds; the reported values should be read as point estimates, not stable benchmarks. Variance across seeds was not evaluated in this phase.

Per-class performance. Reported Stage 2 and Stage 3 scores are macro-F1 aggregates only. Per-class precision, recall, and F1 (e.g. for mechanism vs effect, or caution vs warning) were not computed for this submission, and no confusion matrices are included. Given class imbalance and the Stage 2–3 train/inference representation shift, aggregate macro-F1 may hide collapse on individual labels (especially the sparse caution class). Confusion matrices and per-class F1 tables are planned alongside span-consistent retraining and multi-modal ablation studies in future work.

With the full system described, the results can now be interpreted in context. Each metric reflects a specific design decision or data constraint explored in the preceding sections.

Metric	Value	Notes
Stage 1 NER macro-F1 (B+I only)	0.9517	Strong clinical token representations from Bio_ClinicalBERT
Stage 2 interaction-type macro-F1	0.4190	Macro-F1 only; per-class F1 not reported; limited by linguistic subtlety
Stage 3 severity macro-F1	0.3602	Macro-F1 only; per-class F1 not reported; constrained by 23.31% DDInter coverage; caution class especially sparse
DDInter coverage	795 / 3,411 (23.31%)	Naming mismatches across three incompatible drug vocabularies
Skipped class-level mentions	539	class_routing exclusions (e.g. “beta blocker”)

Stage 1 NER achieves a strong F1 of 0.9517, reflecting Bio_ClinicalBERT’s pre-training on MIMIC-III clinical notes. Stage 2 interaction-type classification (0.4190) is limited by the linguistic subtlety required to distinguish mechanism, effect, advise, and int classes from each other and from non-interactions. Stage 3 severity (0.3602) is most constrained by data: only 795 of 3,411 sentence pairs carry a resolved DDInter severity label, and the caution class is particularly sparse even within that subset. BalancedSoftmaxLoss mitigates but does not eliminate this imbalance.

DDInter Level	Severity Index	Label
(no match / safe negative)	0	safe
minor	1	caution
moderate	2	warning
major / contraindicated	3	danger

12. Limitations & Future Work

12.1 Current Limitations

Data-Linking & Label Constraints

- **Low DDInter Coverage:** Due to strict naming mismatches across disparate drug vocabularies, only 23.31% (795 of 3,411) of sentence pairs successfully resolved to a clinical severity label. This severe data constraint directly limited the Stage 3 classification score.
- **Drug-Class Skipping:** A total of 539 clinical phrases were skipped using `class_routing` (e.g., “beta blocker”) because the pipeline cannot yet automatically map therapeutic categories down to specific ingredients.

Architecture & Evaluation Gaps

- **Train/inference distribution shift (Stages 2–3).** The `drug_fusion` MLP was trained with [CLS] as the BERT drug feature but is applied at inference to span mean-pooled vectors. Because these representations are not equivalent, the learned projection may not transfer faithfully to live inputs. This is not a minor suboptimality: it undermines the validity of interaction and severity outputs from the fine-tuned heads until training is aligned with inference (span-consistent datasets). NER (Stage 1) and KG/Lipinski lookups are unaffected; the pair-level classification path is the main concern.
- **Unverified multi-modal contribution.** MedGuard’s central design claim is that KG embeddings and Lipinski features improve drug-pair understanding alongside clinical text. No ablation was run (text-only vs +KG vs +Lipinski vs full 901-d fusion). Under the CLS/span mismatch, side modalities may be compensating for—or masking—distorted BERT drug features; their marginal benefit is unknown and must not be inferred from end-to-end F1 alone.
- **Single-run metrics.** Reported F1 scores are from one training run without seed replication or confidence intervals (see Section 11). Stage 3 F1 in particular may be unstable given label sparsity.
- **Modality Sparsity:** If a drug is missing from `knowledge_graph.pkl` or the Lipinski data table, the API assigns zero-tensors and switches the coverage flags to false. The model still attempts a text-based prediction instead of gracefully refusing service.
- **Linguistic Fine-Graining:** The Stage 2 head struggled with the extreme semantic subtlety required to differentiate fine-grained linguistic classes within short sentences.

12.2 Future Work

This phase establishes a solid multi-modal inference core, entity-linking pipeline, and REST API. To move MedGuard from an academic prototype toward a practical medication assistant, the following targeted enhancements are planned for the next development cycle:

Patient Application Core

- **Profile-Centric Workflow:** Transition from the single-pair web demo to a React + TypeScript frontend that manages basic local patient profiles (active medication lists, dosages, and schedules).
- **Real-Time Check:** Trigger automated interaction queries against the full profile whenever a user inputs a new prescription.

Model & Data Enhancements

- **Span-Consistent Training (highest priority):** Modify the Stage 2 and 3 training datasets to pass explicit drug entity tokens into the forward() pass to match the exact span mean-pooling approach used at runtime inference. Re-evaluate interaction and severity F1 only after this fix; current Stage 2/3 metrics should not be treated as definitive until then.
- **Expanded Vocabulary Mapping:** Boost Stage 3 data diversity beyond the current 23.31% bottleneck by integrating additional DDInter letter-code files, adding basic fuzzy name matching, and implementing drug-class-to-ingredient expansion.
- **Multi-modal ablation (required for scientific claim):** Train or evaluate variants with KG and Lipinski disabled independently on the same splits, after span consistency is fixed, to measure whether each modality improves macro-F1 or calibration.
- **Per-class evaluation:** Report per-class F1 and confusion matrices for Stages 2 and 3 after span-consistent training, to identify which labels drive low macro-F1.

Verification & Deployment

- **System Testing:** Implement an automated pytest suite for core data processors and API routes to ensure code stability.

13. Conclusion

MedGuard demonstrates that a multi-modal, multi-task framework can successfully integrate clinical text, relational knowledge-graph structures, and molecular chemistry into a single drug-drug interaction detection system. By utilizing a three-stage curriculum training strategy, the architecture optimizes a shared Bio_ClinicalBERT backbone sequentially, preventing high-performing tasks like Named Entity Recognition from dominating gradient descent and harming the more complex classification heads.

The project meets its core engineering objectives by delivering a fine-tuned encoder, an algorithmic six-step DictionaryFirstLinker to handle cross-vocabulary mapping, and a live FastAPI backend supported by a conversational agent layer.

Experimental results show strong NER performance (macro-F1 0.9517 on B-DRUG and I-DRUG) but weaker interaction-type (macro-F1 0.4190) and severity (macro-F1 0.3602) scores. The severity result is constrained by DDInter linkage (23.31% coverage); both classification stages are further affected by reporting from a single training run without seed variance. Most critically, Stages 2–3 were trained with [CLS] drug features while inference uses span pooling, introducing a train/serve shift that may limit the interpretability of current interaction and severity metrics until span-consistent training is completed.

This phase validates the data pipeline, entity linking, multi-modal architecture, and deployable API. It does not yet validate that the full fusion design improves performance over text alone—that requires ablation and aligned training. MedGuard remains an academic demonstration only; the established codebase and honest evaluation provide the foundation for Semester 2 work (patient-facing application, ablations, and stricter evaluation).

References

Models & methods

Alsentzer, E., Murphy, J. R., Boag, W., Weng, W.-H., Jin, D., Naumann, T., & McDermott, M. (2019). Publicly available clinical BERT embeddings. In Proceedings of the 2nd Clinical Natural Language Processing Workshop (pp. 72–78). Association for Computational Linguistics. (Pre-trained model: emilyalsentzer/Bio_ClinicalBERT on Hugging Face.)

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In Proceedings of NAACL-HLT 2019. (Base architecture underlying Bio_ClinicalBERT; optional but strengthens the model lineage.)

Grover, A., & Leskovec, J. (2016). node2vec: Scalable feature learning for networks. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (pp. 855–864).

Menon, A. K., Jiang, X., Sharma, S., Bauer, J., & Narayanan, V. (2021). Long-tail learning via logit adjustment. International Conference on Learning Representations (ICLR).

Datasets

Herrero-Zazo, M., Segura-Bedmar, I., Martínez, P., & Declerck, T. (2013). The DDI corpus: An annotated corpus with pharmacological substances and drug–drug interactions. *Journal of Biomedical Informatics*, 46(5), 914–920. (DDI Corpus 2013 / SemEval-2013 Task 9 — Stages 1–2 training data.)

Wishart, D. S., Feunang, Y. D., Guo, A. C., Lo, E. J., Marcu, A., Grant, J. R., ... & Wilson, M. (2018). DrugBank 5.0: A major update to the DrugBank database for 2018. *Nucleic Acids Research*, 46(D1), D1074–D1082. Full XML accessed under an academic licence following registration at <https://go.drugbank.com/> (version 5.0). (Drug names, synonyms, interactions, KG seed, Lipinski compound IDs.)

Xiong, G., Wang, Y., Chen, Y., Chen, K., Han, D., & Chen, J. Y. (2022). DDInter: An online drug–drug interaction database. *Nucleic Acids Research*, 50(D1), D1200–D1207. (Severity labels for Stage 3 — `ddinter_code_*.csv`.)

Lipinski, C. A., Lombardo, F., Dominy, B. W., & Feeney, P. J. (1997). Experimental and computational approaches to estimate solubility and permeability in drug discovery and development settings. *Advanced Drug Delivery Reviews*, 23(1–3), 3–25. (Lipinski Rule of Five descriptors; values from DrugBank-derived `DB_compounds_lipinski.csv`.)